



MCP最佳落地实践

动手打造你的第一个AI本地工具



AI的“玻璃樽”困境

拥有强大的推理能力，
但对你的世界一无所知。

包含着最即时、最个人化的
数据和工具。



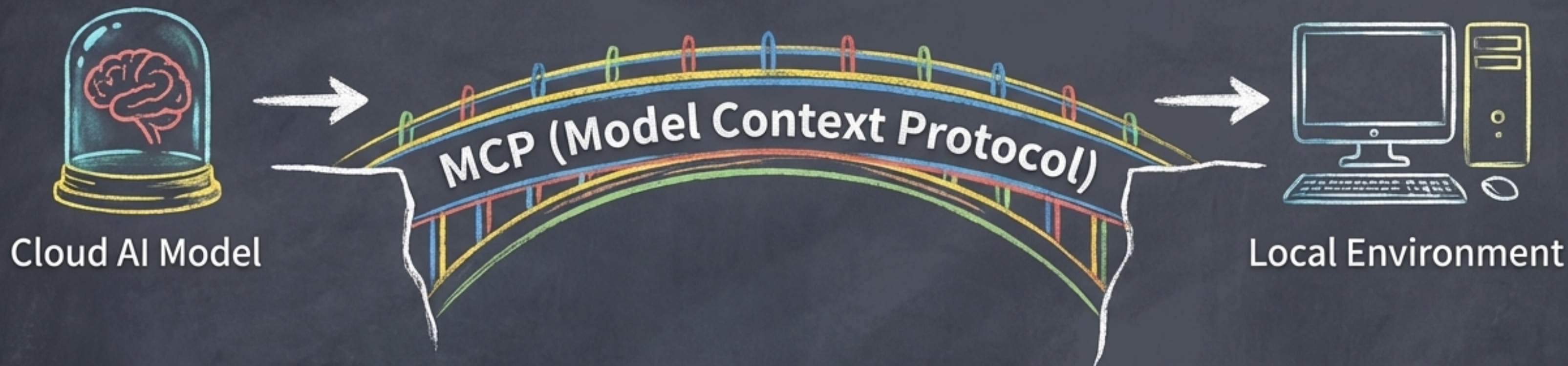
Cloud AI Model



Local Environment

我们如何打破这堵墙？

MCP：连接AI与现实世界的桥梁

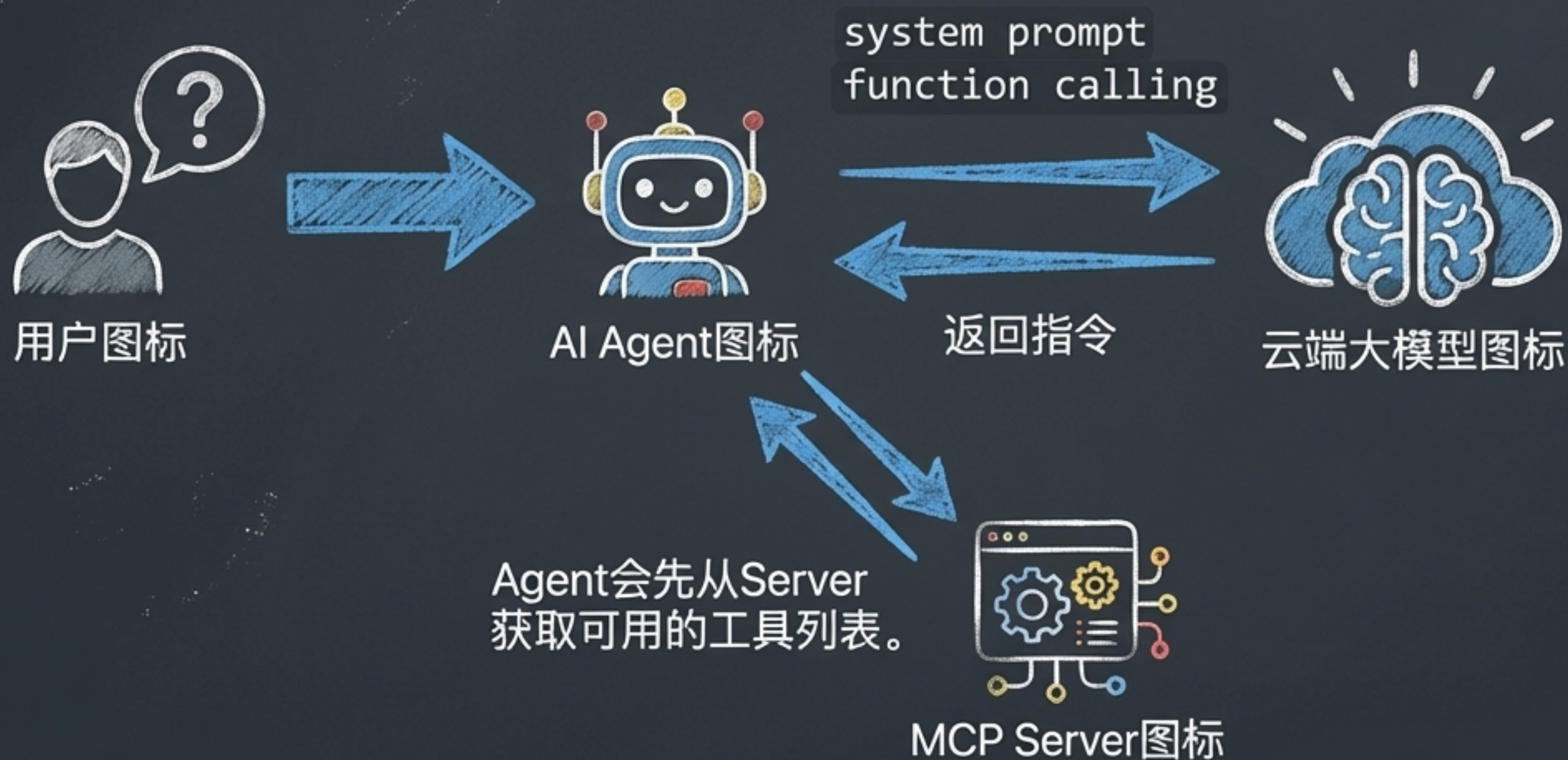


什么是MCP?

- 它是一种标准协议，允许AI模型安全、可靠地访问用户提供的本地数据和工具。
- 把它想象成AI的“感官”，让它能够“看到”和“操作”你的本地环境。



MCP工作原理解析



MCP Server就像一个工具箱目录。AI Agent先查阅目录，然后告诉大模型“我们有这些工具可以用”。

我们的“建造”蓝图

1

第一站：工具铸造 (`tools.py`)

- 编写一个能获取本机系统信息的Python函数。
- 重点：如何写出AI能“看懂”的文档字符串 (Docstring) 。

2

第二站：服务启动 (`main.py`)

- 使用 `fastmcp` 库快速搭建MCP服务器。
- 将我们铸造的工具“注册”到服务器上。



3

第三站：实战连接 (Claude Desktop)

- 配置客户端，让AI Agent真正用上我们的本地工具。



Step 1: 搭建项目框架

WORKPLACE_WQQ/

├── main.py

├── tools.py

└── main.py

****tools.py****: 我们的工具库。所有能被AI调用的本地函数都放在这里。可以把它想象成我们的“武器库”。

****main.py****: MCP服务器的启动入口。它负责加载工具，并启动服务，等待AI Agent的连接。

Step 2: 铸造核心工具 `get\_host\_info`

```
# tools.py
```

```
import platform  
import psutil  
import subprocess  
import json
```

利用强大的Python标准库和第三方库获取系统信息。

```
def get_host_info() -> str:  
    """get host information  
    Returns:  
        str: the host information in JSON string  
    """
```

```
    info: dict[str, str] = {  
        "system": platform.system(),  
        "release": platform.release(),  
        "machine": platform.machine(),  
        "processor": platform.processor(),  
        "memory_gb": str(round(psutil.virtual_memory().total / (1024**3), 2)),  
    }
```

将收集到的系统信息结构化地存放在字典中，方便后续处理。

```
# ...
```

```
    return json.dumps(info, indent=4)
```


关键一步：为AI编写“使用说明书”

```
def get_host_info() -> str:  
    """get host information  
  
    Returns:  
        str: the host information in  
        JSON string  
    """  
    # ... function body
```

“Docstring是AI理解工具功能的唯一途径！描述越清晰、越准确，AI调用它的成功率就越高。”

- ✓ **功能描述：**清晰说明函数做什么。（get host information）
- ✓ **参数说明：**（本例无）如果有参数，要详细解释每个参数的含义和类型。
- ✓ **返回说明：**描述返回值的格式和内容。（the host information in JSON string）

Step 3: 三步启动MCP服务

```
# main.py
from mcp.server.fastmcp import FastMCP
import tools

def main():
    # 1. 初始化服务, 并给它一个描述
    mcp = FastMCP("host info mcp")
    # 2. 注册我们的工具
    mcp.add_tool(tools.get_host_info)
    # 3. 启动! 使用stdio模式
    mcp.run("stdio")

if __name__ == "__main__":
    main()
```

①

① **初始化**: 创建MCP服务器实例。

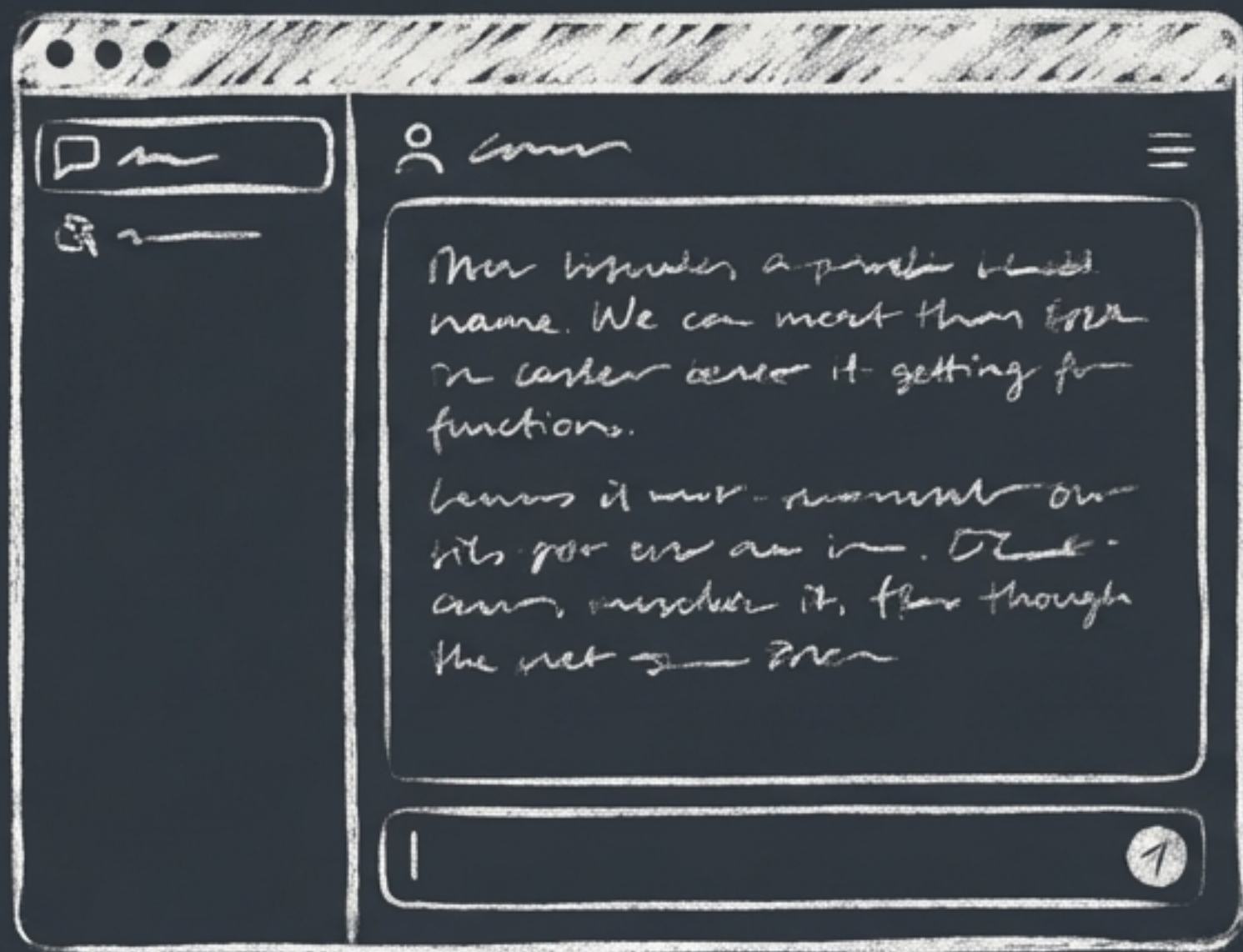
②

② **注册工具**: 把`get\_host\_info`函数“装载”到服务器上。

③

③ **运行服务**: 以标准输入输出 (stdio) 模式启动, 等待客户端连接。

Step 4: 连接AI Agent (Claude Desktop)



设置



开发者



编辑配置

我们的MCP Server已经准备就绪，现在需要告诉Claude Desktop应用如何找到并运行它。这需要通过修改一个JSON配置文件来完成。

核心配置: `claude\_desktop\_config.json`

```
{
  "mcpServers": {
    "hostInfoMcp": {
      "command": "C:\\\\Users\\...\\anaconda3\\envs\\wqq\\python.exe",
      "args": [
        "D:\\\\Workspace_WQQ\\AI_MCP\\main.py"
      ]
    }
  }
}
```

我们为这个服务起的名字，可以自定义。

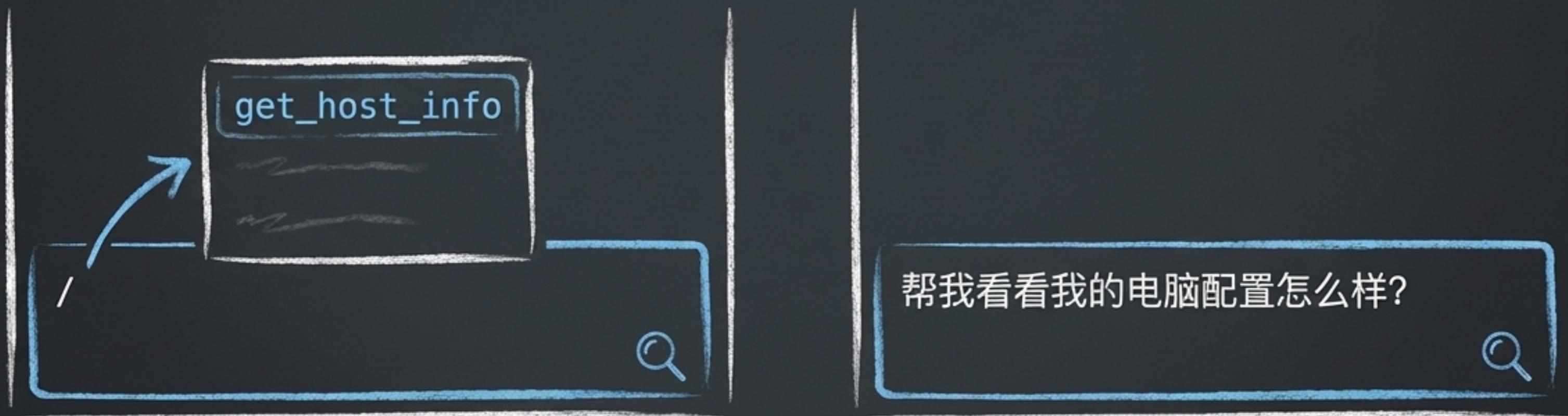
`command`

- **作用:** 指定用来执行我们脚本的程序。
- **最佳实践:** 强烈建议指向一个虚拟环境的Python解释器路径! 这样可以隔离依赖, 避免版本冲突。

`args`

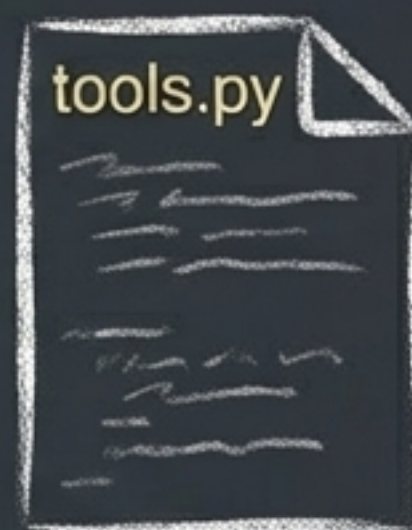
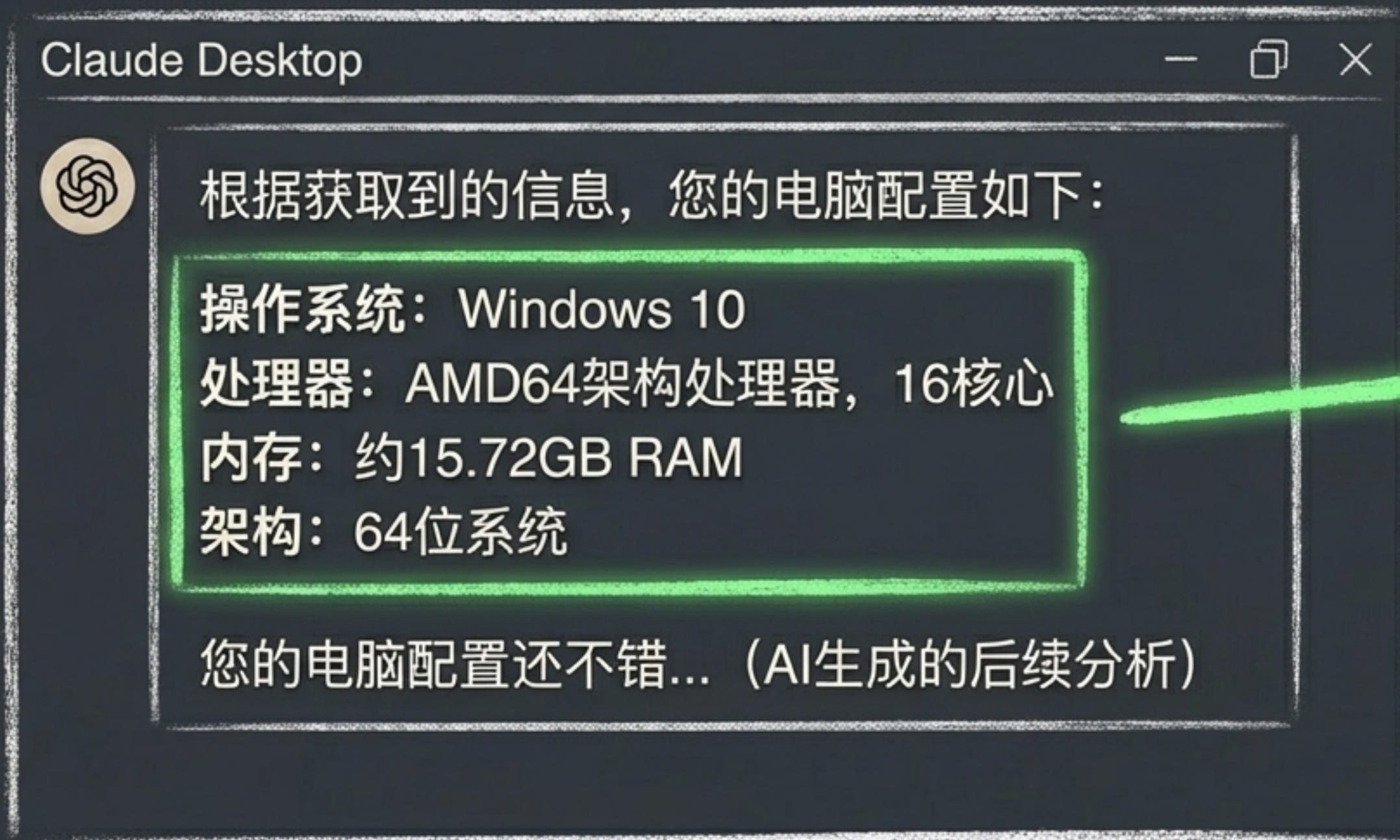
- **作用:** 传递给`command`的参数列表。
- **内容:** 这里是我们`main.py`脚本的绝对路径。

见证奇迹的时刻：启动与测试



配置完成后，重启Claude Desktop。当你输入 / 时，如果能看到 `get_host_info`，就说明我们的MCP服务已经被成功识别了！现在，让我们向它提问。

成功！AI调用了我们的本地代码



这些数据100%来自
我们本地运行的
Python脚本！

全流程回顾：从代码到答案



MCP开发最佳实践清单



清晰的文档 (Clear Docstrings)

AI的“眼睛”，你写得越清楚，它看得越明白。



单一职责原则 (Single Responsibility)

让每个工具只做一件事，并把它做好。这能极大提高AI选择工具的准确性。



环境隔离 (Environment Isolation)

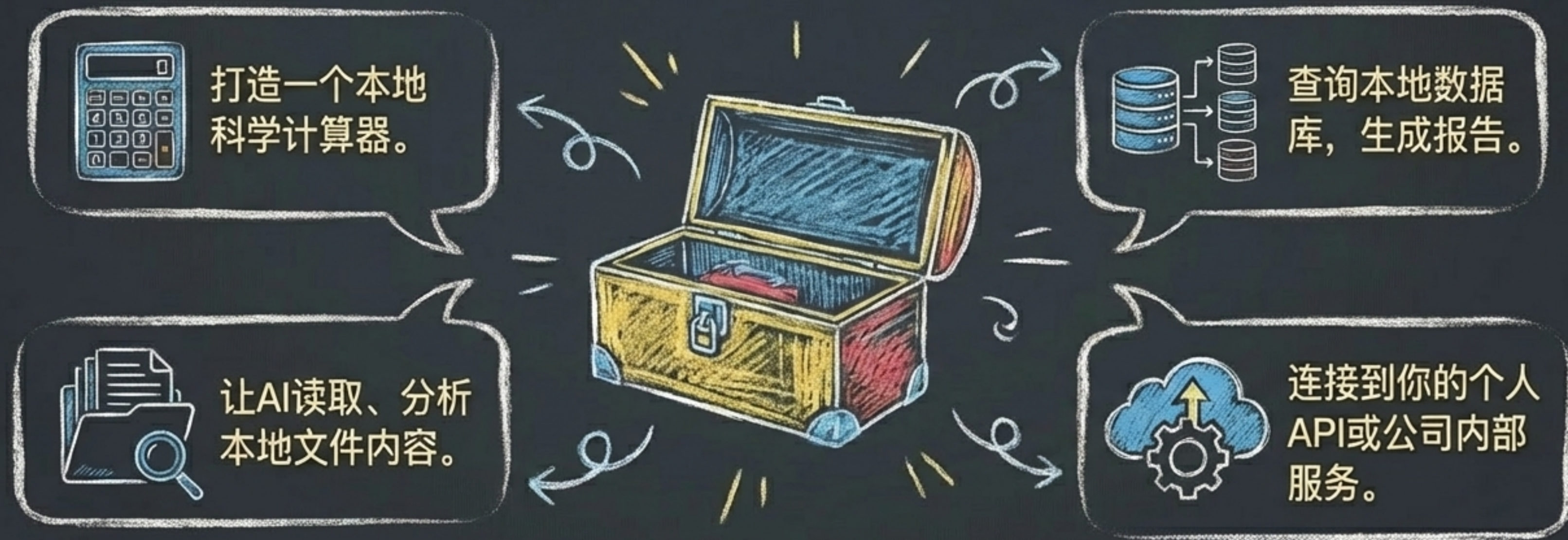
永远使用虚拟环境 (venv, conda) 来管理项目依赖，这是专业开发的基石。



安全第一 (Security First)

警惕！不要创建可以执行任意代码或修改关键系统文件的工具，除非你完全清楚其风险。

你的工具箱，未完待续...



现在，轮到你了。去创造能真正赋能你工作流的AI工具吧！